

Matrix Multiplication

Setup

Example

- A is 4x6
- B is 6x4
- C is 4x4
- $m = 4, k = 6, n = 4$
- $\text{TILE_SIZE} = 2$
- $\text{BLOCK_SIZE} = 2 \times 2$

A (4x6)

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)

1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6

C (4x4)

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Kernel Launch

A (4x6)

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)

1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6

C (4x4)

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

```
mysgemm<<<dim_grid, dim_block>>>(m, n, k, A, B, C);
```

- C is m×n (4x4)→ the grid is shaped directly after C: **$\text{ceil}(m/\text{TILE_SIZE}) \times \text{ceil}(n/\text{TILE_SIZE}) = 2 \times 2 = 4 \text{ blocks}$** .
- Each **block** holds $\text{TILE_SIZE} \times \text{TILE_SIZE} = 2 \times 2 = 4 \text{ threads}$, giving $4\text{blk} \times 4\text{threads/blk} = 16 \text{ threads total}$.
- The grid is designed so that every cell of C maps to exactly one thread, each thread computes the full dot product for its assigned output element*. (In this example, C has 16 elements, we launched 16 threads)

*: The grid is designed so that at least one thread exists per cell of C. When m and n are divisible by TILE_SIZE the mapping is exactly one-to-one. Otherwise, the ceiling division creates more threads than output elements — extra boundary threads are spawned but suppressed by the guard if (row < m && col < n) and write nothing to C.

Lets focus on **block = 0**, **thread = (0,0)**

A (4x6)

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)

1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6



SHARED MEM - AS[2][2]

?	?
?	?

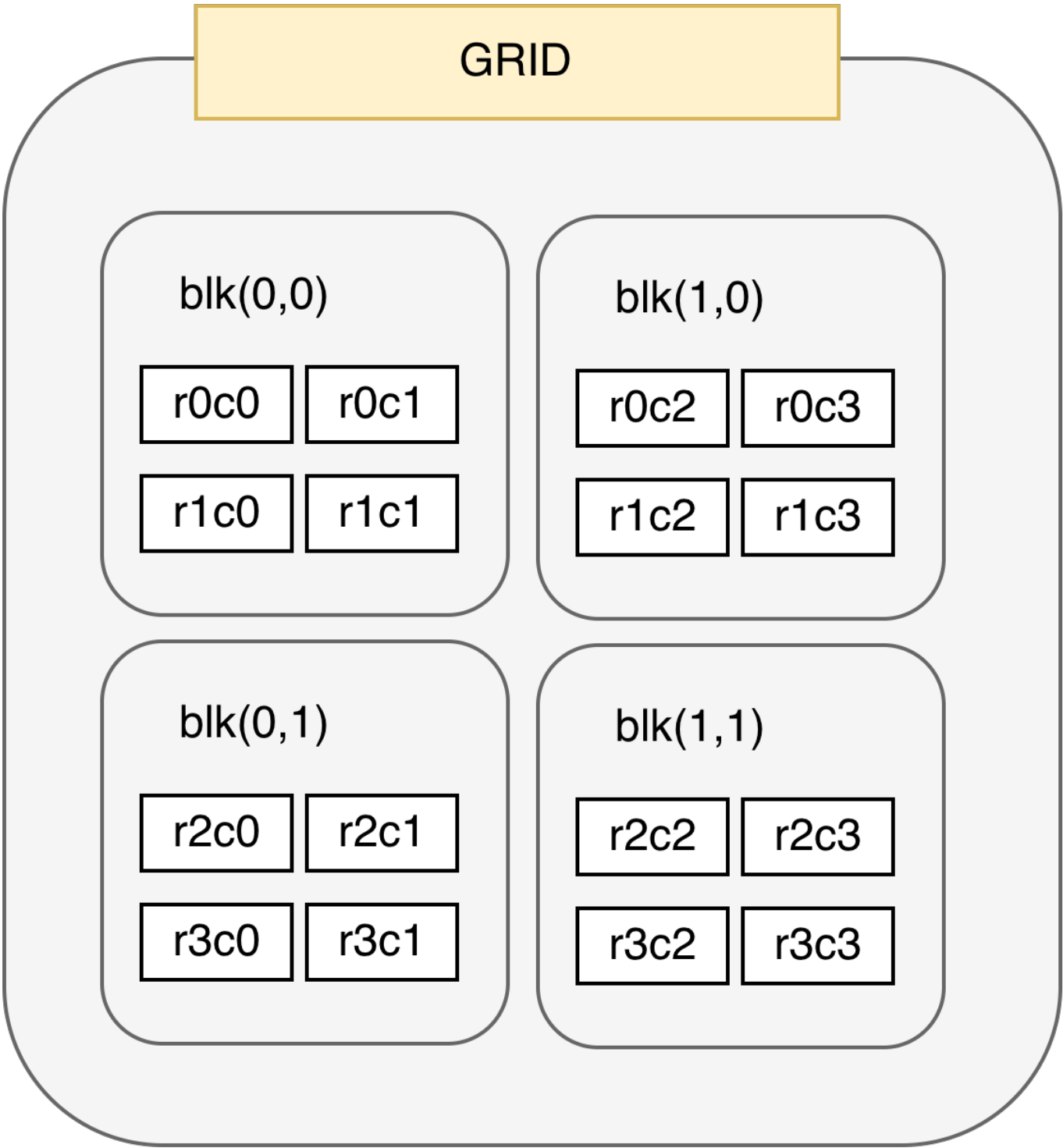
SHARED MEM - BS[2][2]

?	?
?	?

C (4x4)

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Each output element of C is a dot product of a row of A and a column of B. That row has 6 elements. We can only load 2 elements at a time (TILE_SIZE=2). So **each thread** needs 3 rounds to see all 6 elements, for example for the threadIdx = 0, round 1 (elements 1–2), round 2 (elements 3–4), round 3 (elements 5–6). That is what the outer loop does. Lets see the process for a single thread in the next slides.



Lets focus on $\text{block} = 0$, $\text{thread} = (0,0)$

A (4x6)

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)

1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6



SHARED MEM - AS[2][2]

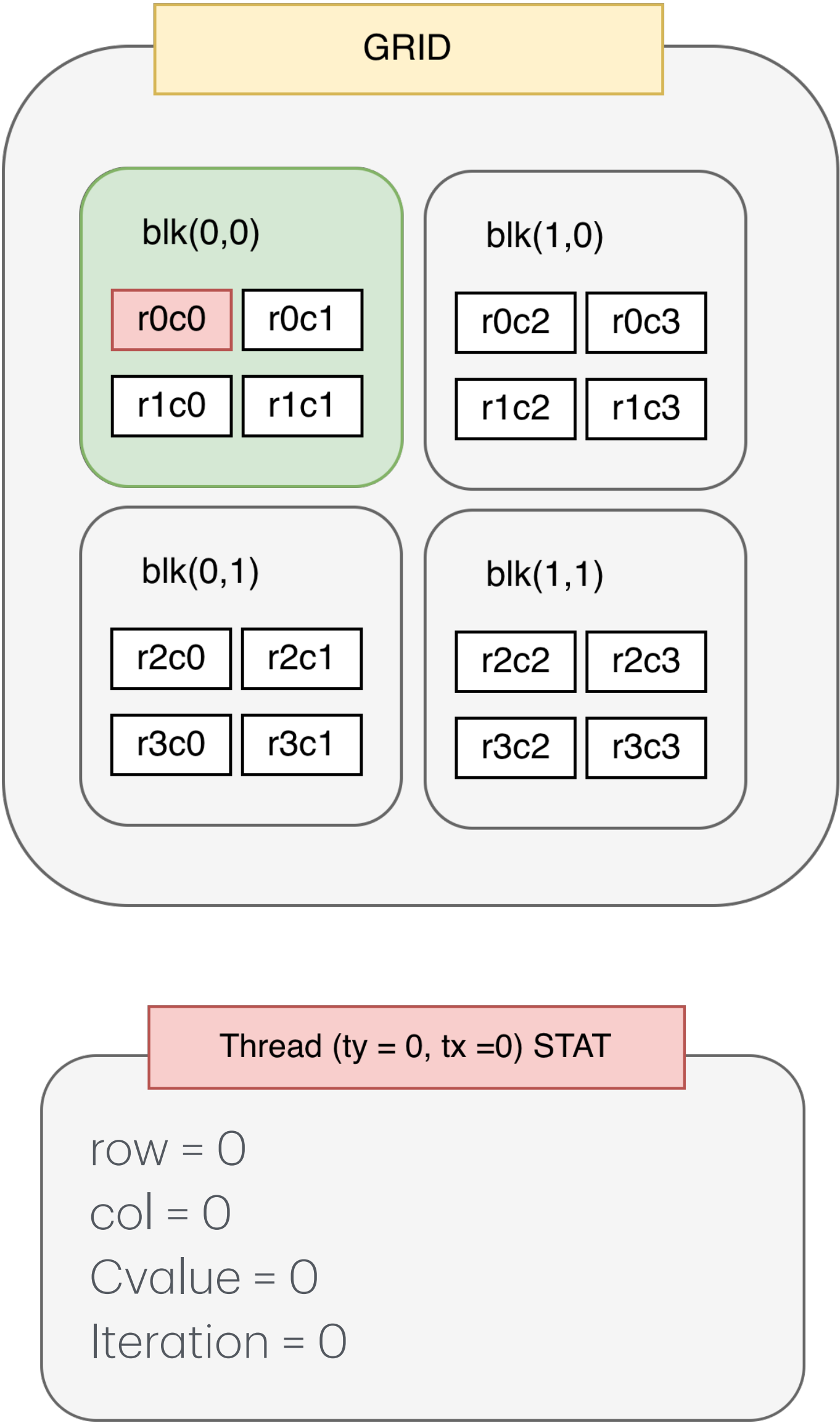
1	2
7	8

SHARED MEM - BS[2][2]

1	0
0	1

C (4x4)

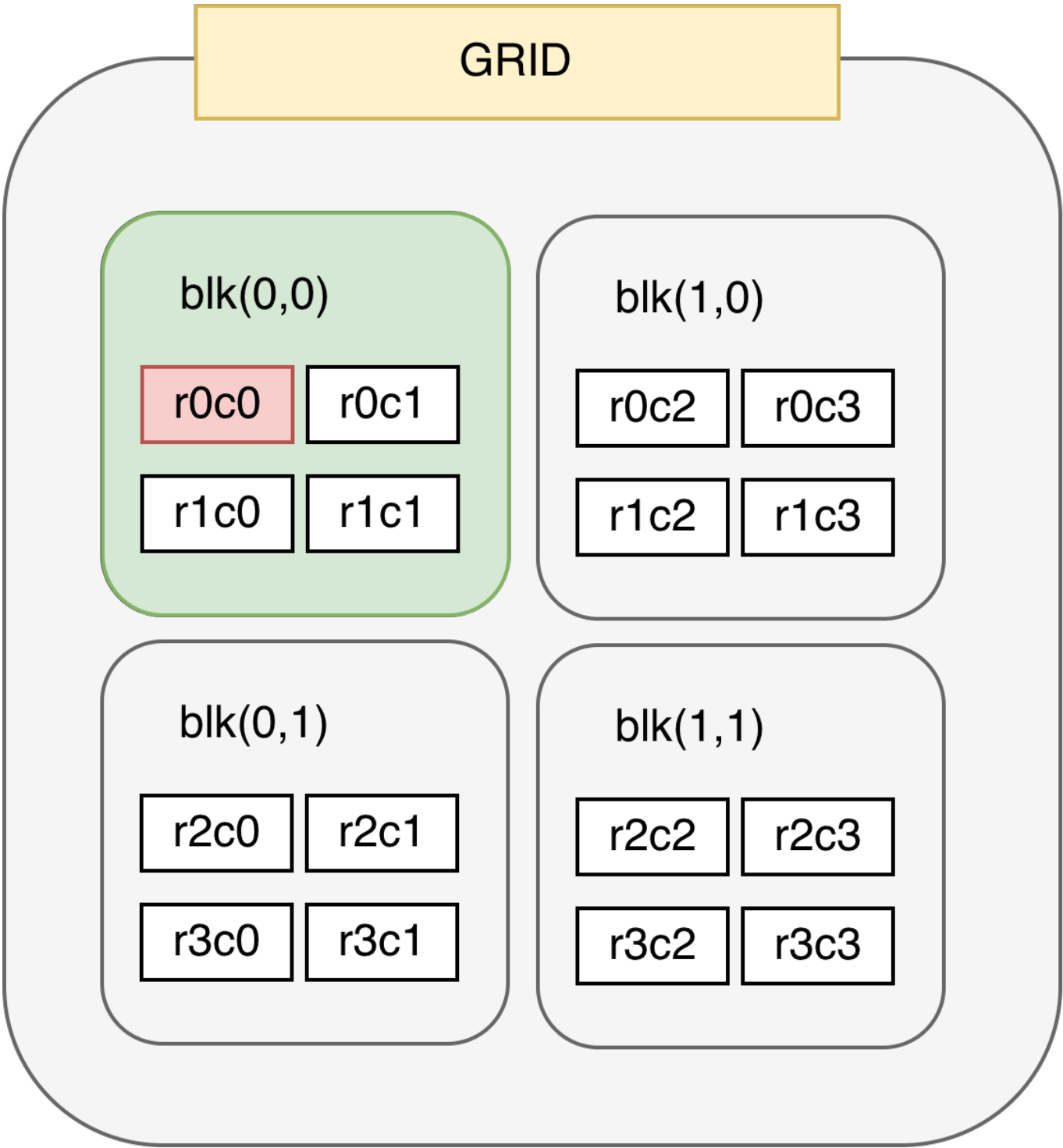
(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)



Lets focus on $\text{block} = 0$, $\text{thread} = (0,0)$

A (4x6)						B (6x4)			
1	2	3	4	5	6	1	0	2	0
7	8	9	10	11	12	0	1	0	2
13	14	15	16	17	18	3	0	4	0
19	20	21	22	23	24	0	3	0	4
						5	0	6	0
						0	5	0	6

SHARED MEM - AS[2][2]		1	2
		7	8
SHARED MEM - BS[2][2]		1	0
		0	1



C (4x4)			
(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

$(1 \times 1) + (2 \times 0) = 1$

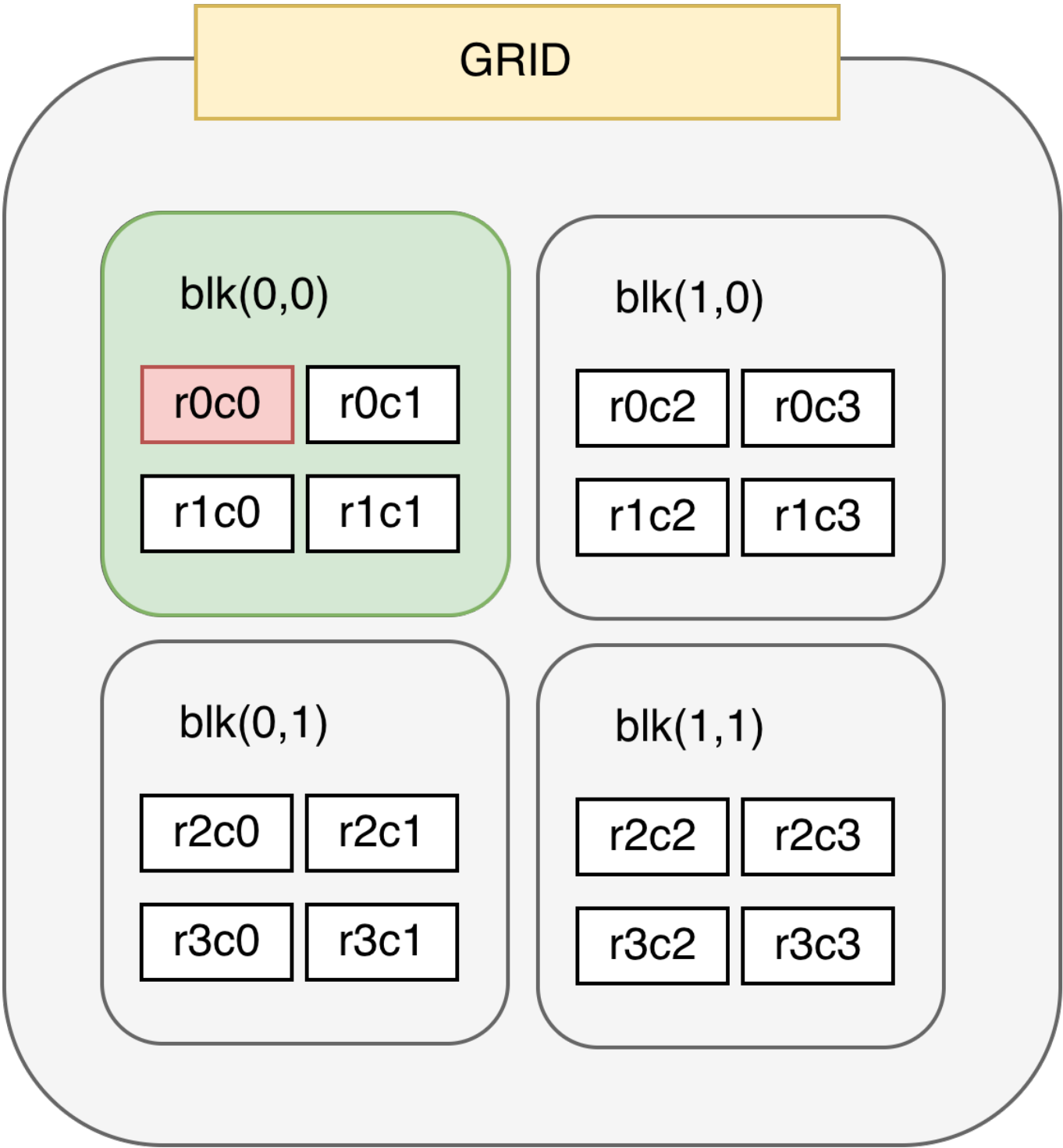
Thread (ty = 0, tx =0) STAT

row = 0
col = 0
Cvalue = 1 (changes)
Iteration = 0

Lets focus on $\text{block} = 0$, $\text{thread} = (0,0)$

A (4x6)						B (6x4)			
1	2	3	4	5	6	1	0	2	0
7	8	9	10	11	12	0	1	0	2
13	14	15	16	17	18	3	0	4	0
19	20	21	22	23	24	0	3	0	4
						5	0	6	0
						0	5	0	6

SHARED MEM - AS[2][2]	
3	4
9	10
SHARED MEM - BS[2][2]	
3	0
0	3



C (4x4)			
(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

$(3 \times 3) + (4 \times 0) = 9$

Thread (ty = 0, tx =0) STAT

row = 0
col = 0

Cvalue = 1 + 9 = 10 (changes)
Iteration = 1 (changes)

Lets focus on $\text{block} = 0$, $\text{thread} = (0,0)$

A (4x6)

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)

1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6



SHARED MEM - AS[2][2]

5	6
11	12

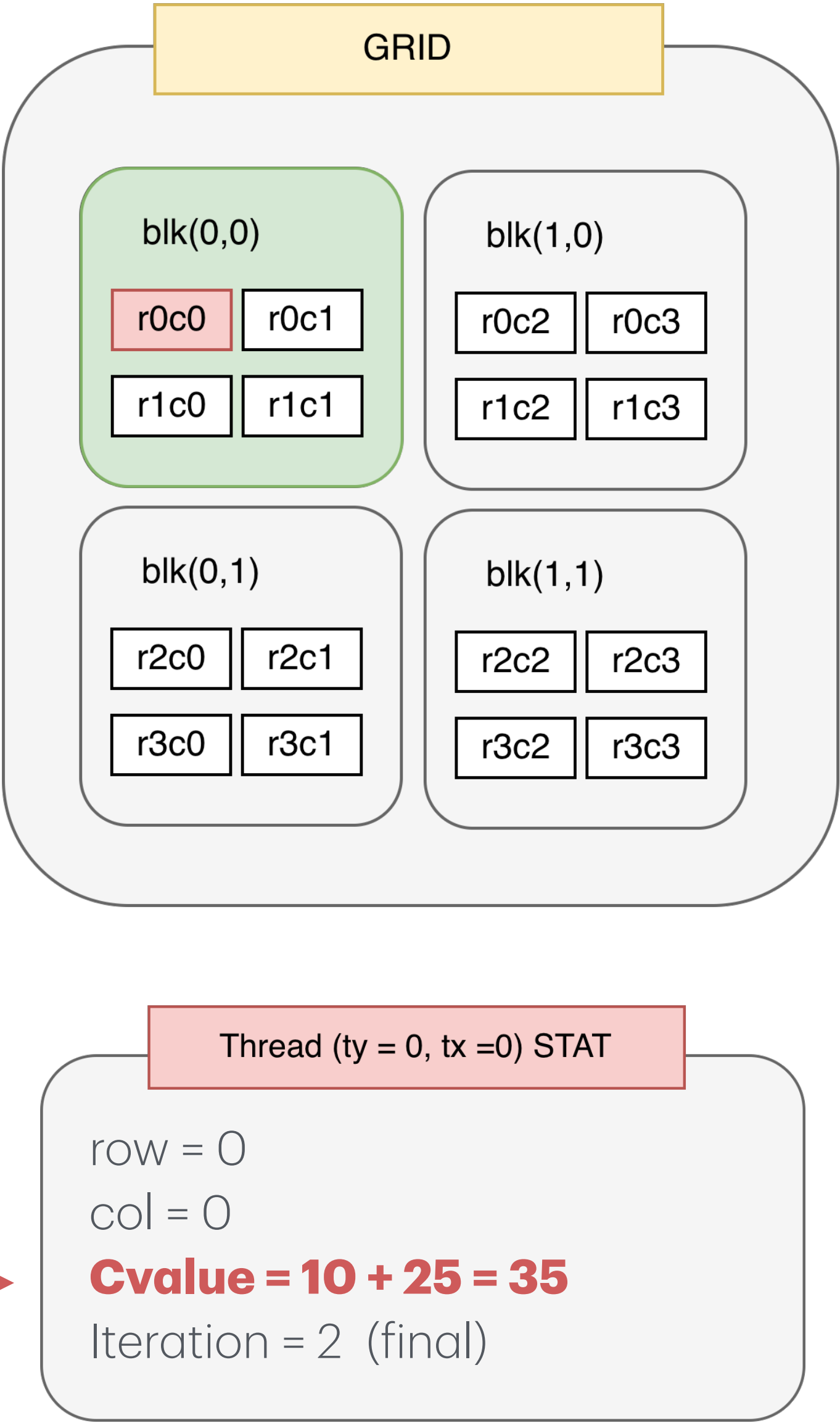
SHARED MEM - BS[2][2]

5	0
0	5

C (4x4)

35	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

$(5 \times 5) + (6 \times 0) = 25$



Lets focus on $\text{block} = 0$, $\text{thread} = (0,0)$

A (4x6)

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)

1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6

SHARED MEM - AS[2][2]

5	6
11	12

SHARED MEM - BS[2][2]

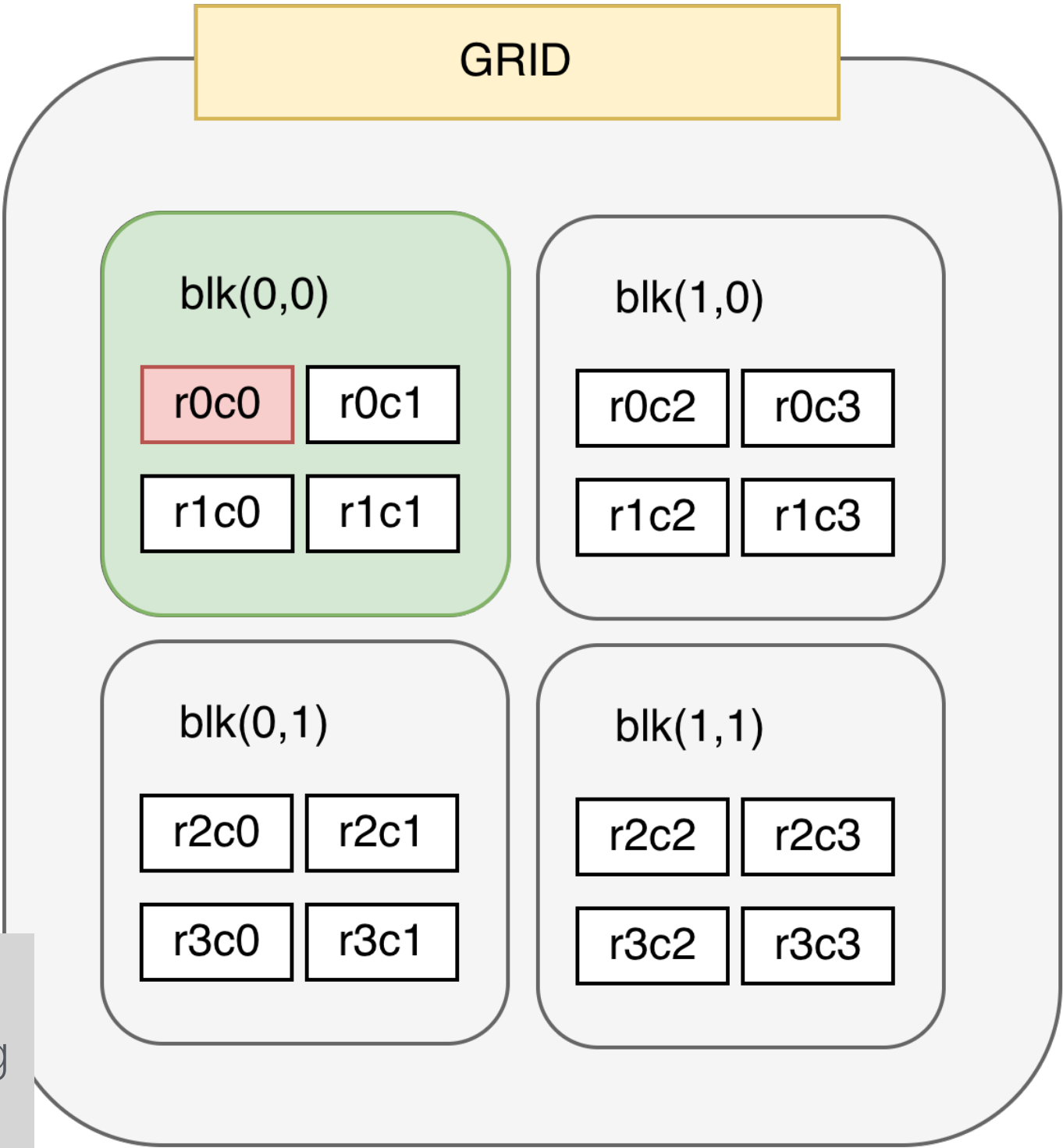
5	0
0	5

?

Thread (0,0) only uses row 0 of AS and col 0 of BS. What is the point of loading the rest of the tile? (In this example, why do we bother loading row 1 and col 1 into shared memory at all?)

C (4x4)

35	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)



Thread (ty = 0, tx =0) STAT

row = 0
col = 0
Cvalue =10 + 25 = 35
Iteration = 2 (final)

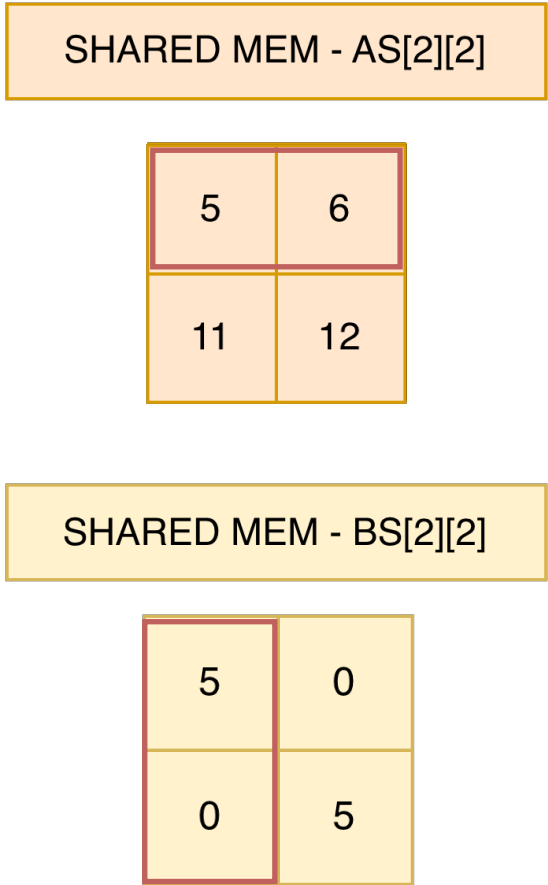
Lets focus on **block = 0**, **thread = (0,0)**

A (4x6)

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)

1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6



?

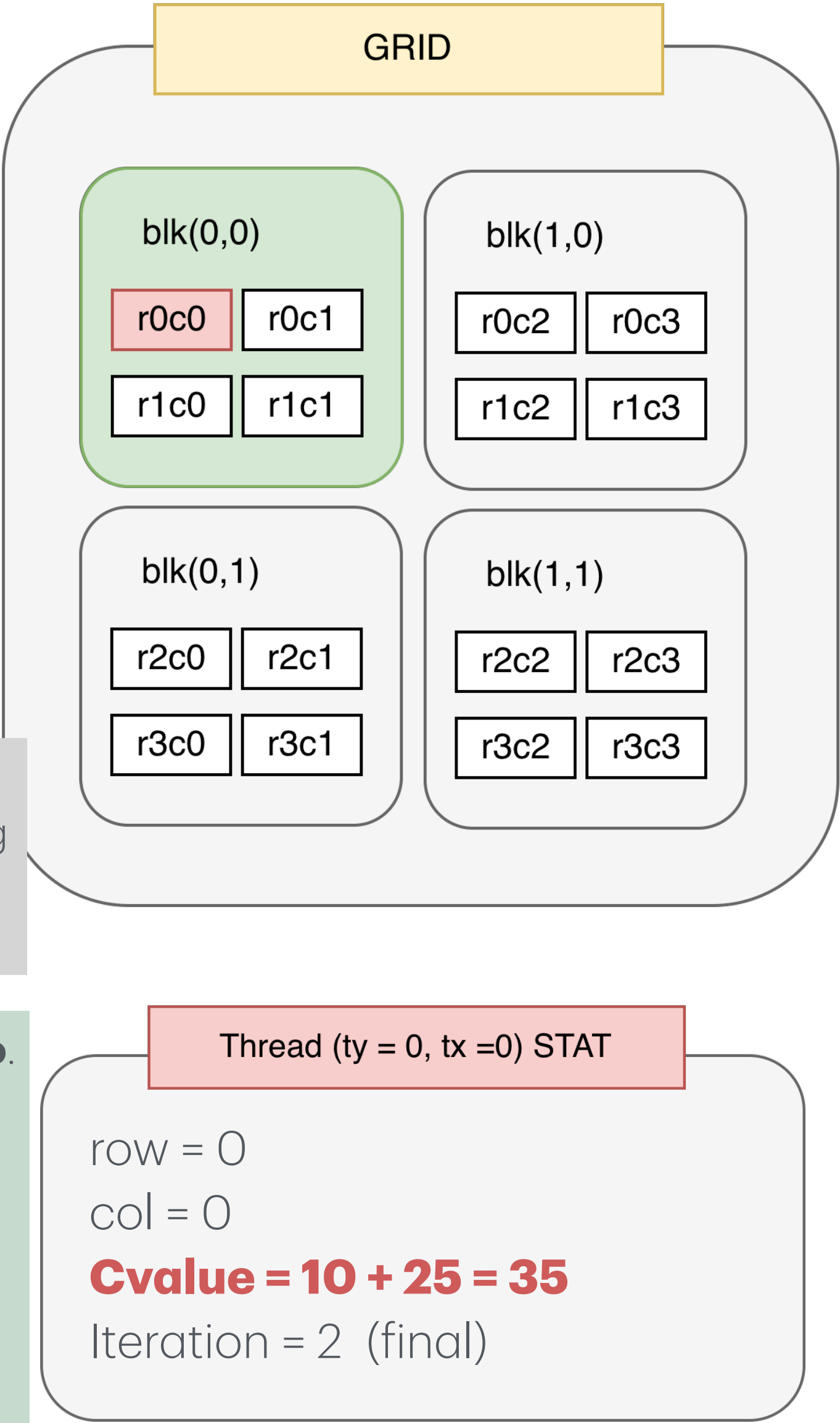
Thread (0,0) only uses row 0 of AS and col 0 of BS. What is the point of loading the rest of the tile? (In this example, why do we bother loading row 1 and col 1 into shared memory at all?)

C (4x4)

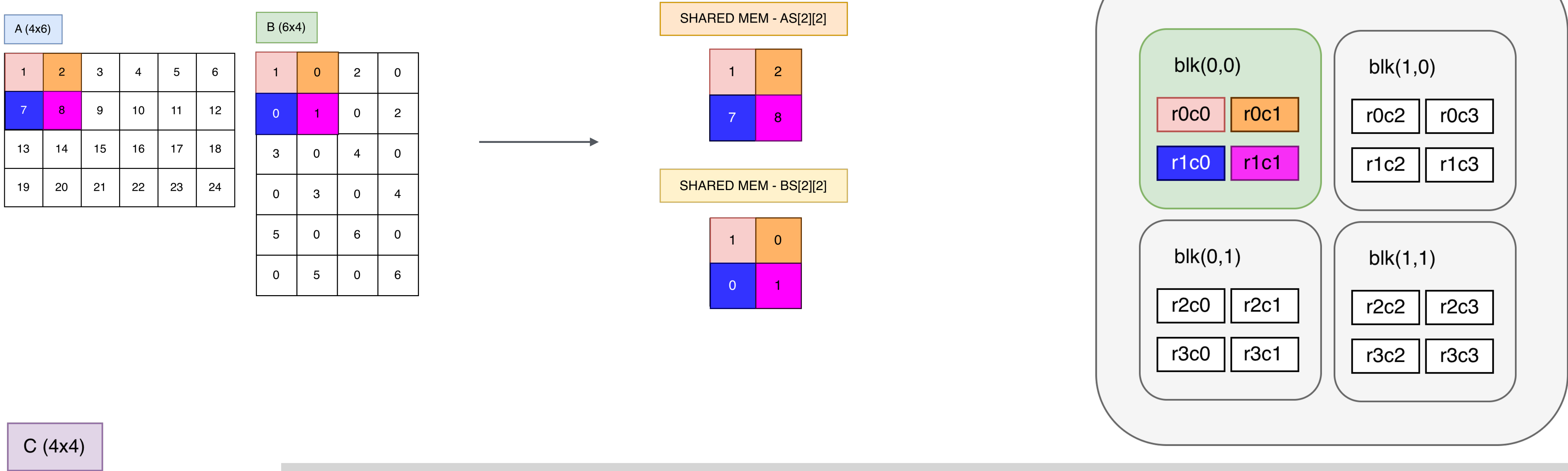
35	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Thread (0,0) does not reuse that data, but its **neighbors in the same block do**. All 4 threads load the tile together, then each picks the row and column it needs.

The point of tiling: without shared memory, each thread would fetch its data independently from slow global memory. With shared memory, **all 4 threads cooperate to load the tile once**. Then each reads its slice from fast shared memory. The data is loaded once, reused by all threads within the same block.



Lets focus on multiple threads within a block — Loading
 $\text{block} = 0$: $\text{thread} = (0,0)$, $\text{thread} = (1,0)$, $\text{thread} = (0,1)$, $\text{thread} = (1,1)$



C (4x4)

(1,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Every thread loads exactly **one element from A**, and **one element from B** and writes into $\text{As}[\text{ty}][\text{tx}]$ and $\text{Bs}[\text{ty}][\text{tx}]$.

Notation: $\text{thread}(\text{tx}, \text{ty})$ inside $\text{blk}(\text{bx}, \text{by})$
 $\text{row} = \text{by} * \text{TILE_SIZE} + \text{ty}$
 $\text{col} = \text{bx} * \text{TILE_SIZE} + \text{tx}$
Example:
thread(1,0) $\rightarrow \text{row} = 0 * 2 + 0 = 0$ $\text{col} = 0 * 2 + 1 = 1 \rightarrow \text{r0c1}$

Lets focus on multiple threads within a block — Loading
block = 0: thread = (0,0), thread = (1,0), thread = (0,1), thread = (1,1)

A (4x6)

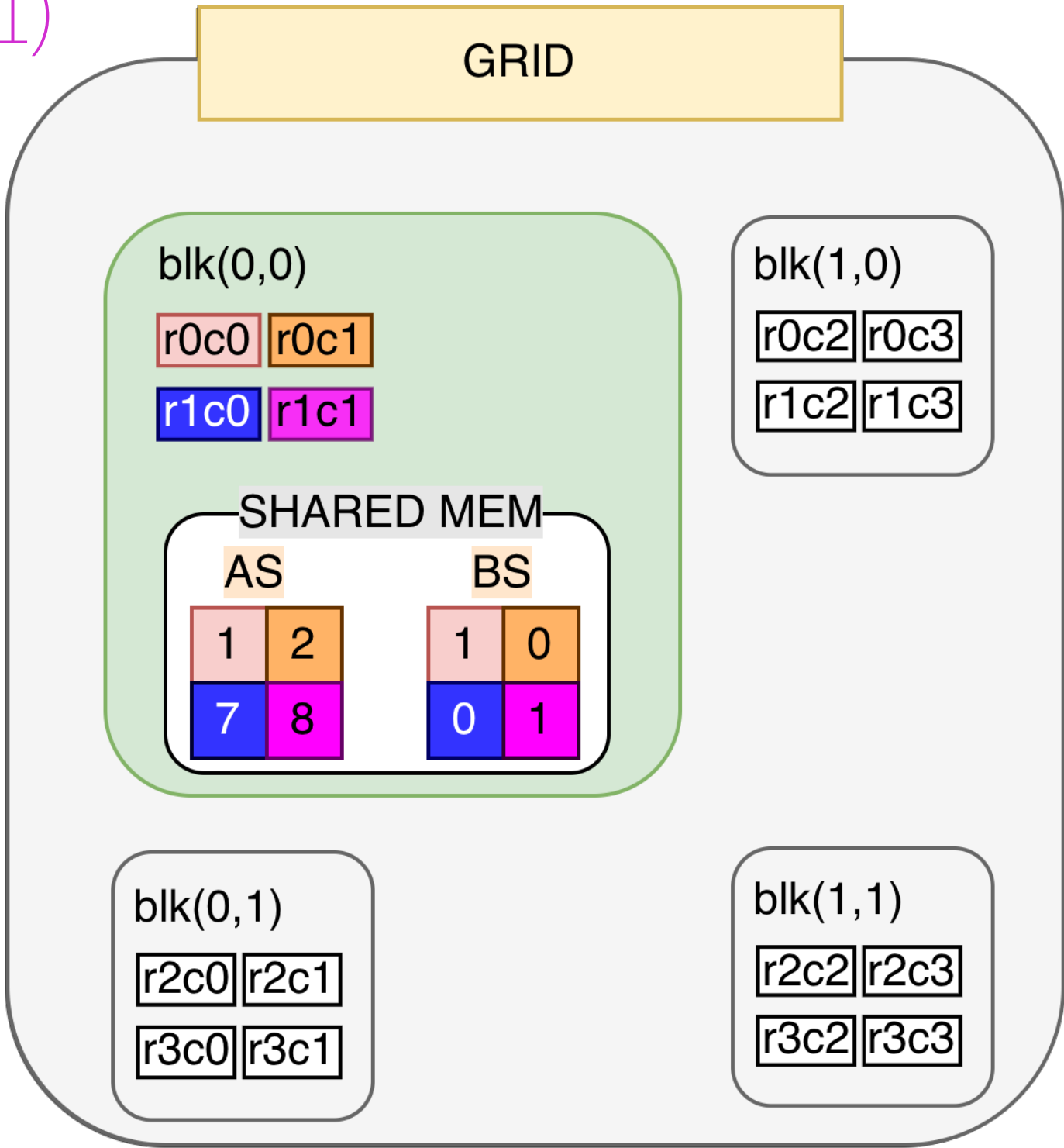
1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)

1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6

C (4x4)

(1,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)



All 4 threads load simultaneously, no single thread does all the work. The tile is filled **cooperatively**, one element per thread for As, and Bs, **all in parallel**. Then `__syncthreads()` makes sure every slot is filled before anyone starts reading from shared memory.

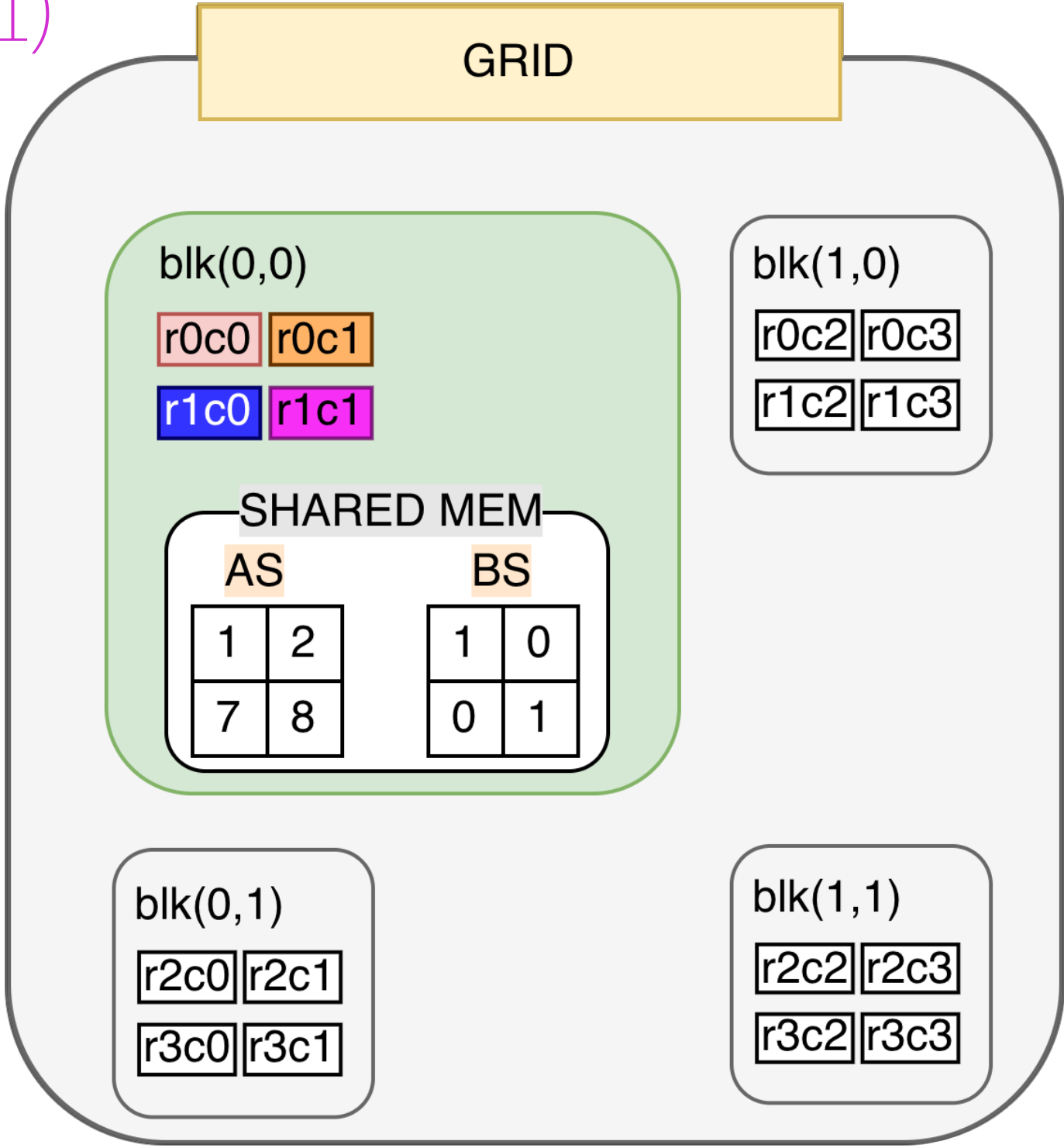
This is the second reason tiling is efficient, not only is the data read from fast shared memory, the loading itself is parallelized across all threads in the block.

Lets focus on multiple threads within a block — Computation
block = 0: thread = (0,0), thread = (1,0), thread = (0,1), thread = (1,1)

A (4x6)					
1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)			
1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6

Iteration = 0



C (4x4)			
(1,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Thread (tx = 0, ty =0) STATE

C[0][0] | row =0, col =0

CValue = 0

Thread (tx = 1, ty =0) STATE

C[0][1] | row =0, col =1

CValue = 0

Thread (tx = 0, ty =1) STATE

C[1][0] | row =1, col =0

CValue = 0

Thread (tx = 1, ty =1) STATE

C[1][1] | row =1, col =1

CValue = 0

Lets focus on multiple threads within a block — Computation
 $\text{block} = 0$: $\text{thread} = (0,0)$, $\text{thread} = (1,0)$, $\text{thread} = (0,1)$, $\text{thread} = (1,1)$

A (4x6)						B (6x4)			
1	2	3	4	5	6	1	0	2	0
7	8	9	10	11	12	0	1	0	2
13	14	15	16	17	18	3	0	4	0
19	20	21	22	23	24	0	3	0	4
						5	0	6	0
						0	5	0	6

Iteration = 0

C (4x4)			
(1,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Thread (tx = 0, ty =0) STATE

C[0][0] | row =0, col =0
1x1 + 2x0 = 1

CValue = 1

Thread (tx = 1, ty =0) STATE

C[0][1] | row =0, col =1
1x0 + 2x1 = 2

CValue = 2

Thread (tx = 0, ty =1) STATE

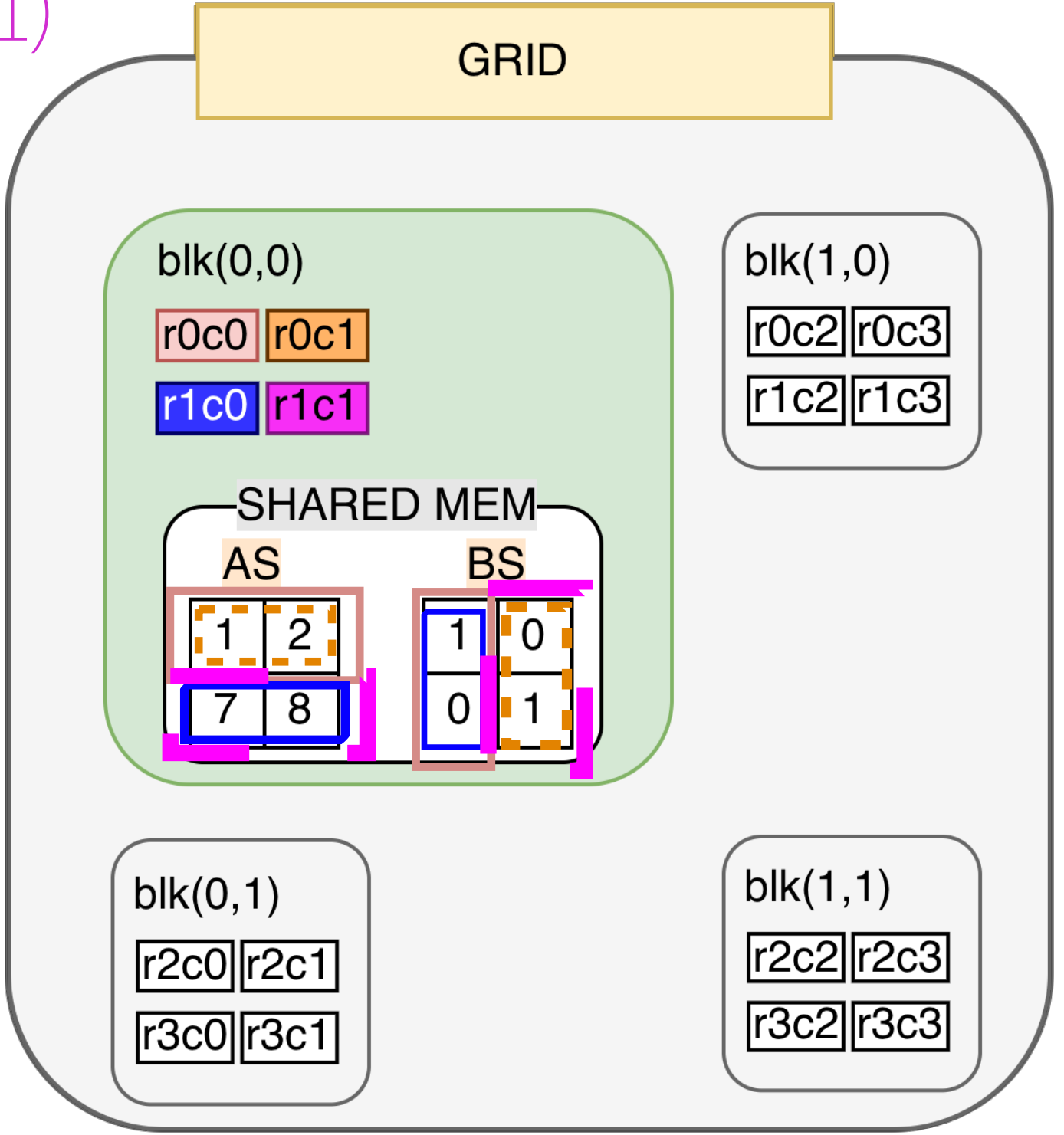
C[1][0] | row =1, col =0
7x1 + 8x0 = 7

CValue = 7

Thread (tx = 1, ty =1) STATE

C[1][1] | row =1, col =1
7x0 + 8x1 = 8

CValue = 8



!

Each thread picks its own row from AS and its own col from BS. They **all run the inner loop simultaneously.**

Lets focus on multiple threads within a block — Computation
 $\text{block} = 0$: $\text{thread} = (0,0)$, $\text{thread} = (1,0)$, $\text{thread} = (0,1)$, $\text{thread} = (1,1)$

A (4x6)

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)

1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6

Load into shared memory

3	4	3	0
9	10	0	3

Iteration = 1

C (4x4)

(1,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Thread (tx = 0, ty =0) STATE

$C[0][0]$ | row =0, col =0

$3 \times 3 + 4 \times 0 = 9$

CValue = 1 + 9 = **10**

Thread (tx = 1, ty =0) STATE

$C[0][1]$ | row =0, col =1

$3 \times 0 + 4 \times 3 = 12$

CValue = 2 + 12 = **14**

Thread (tx = 0, ty =1) STATE

$C[1][0]$ | row =1, col =0

$9 \times 3 + 10 \times 0 = 27$

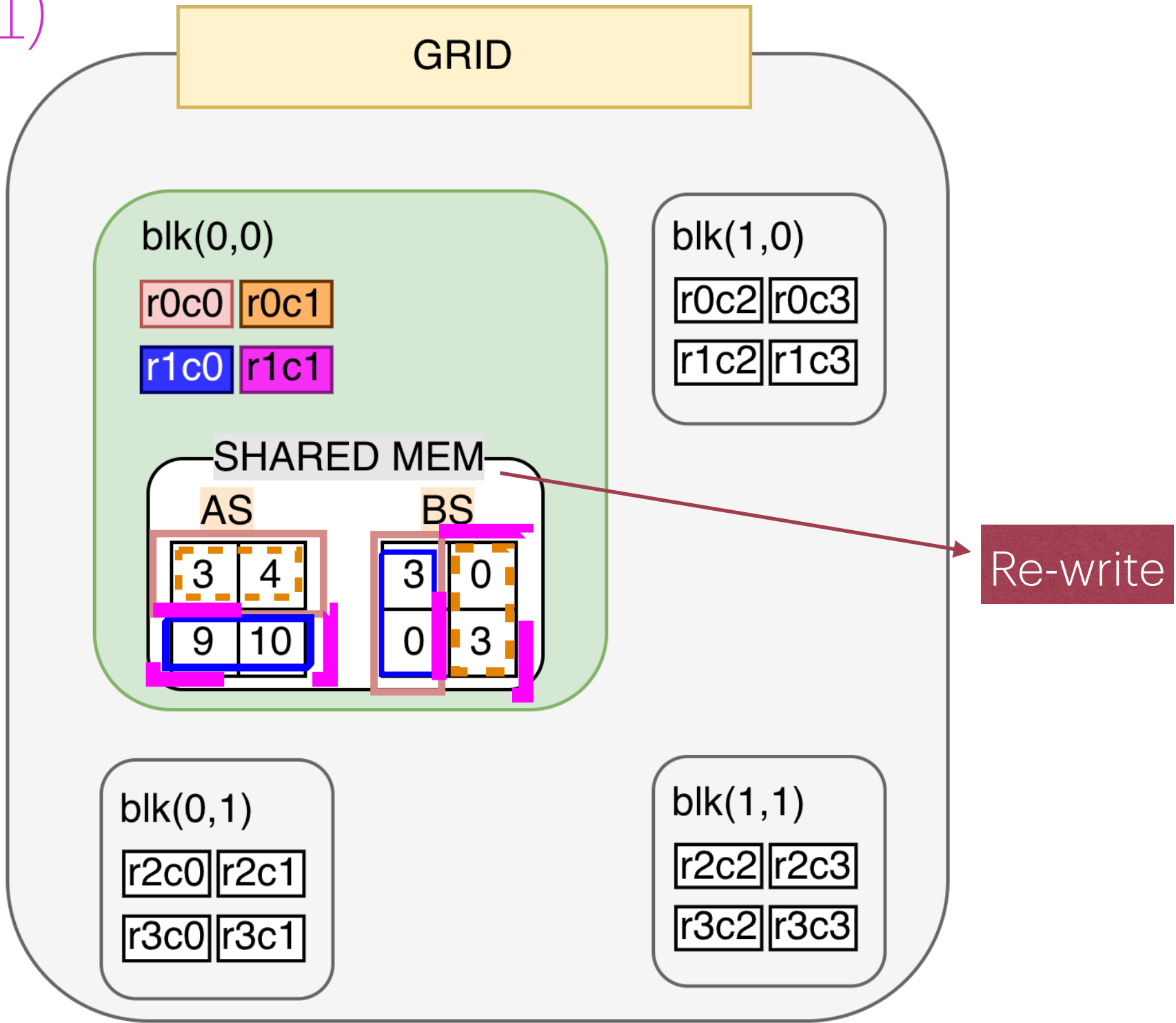
CValue = 7 + 27 = **34**

Thread (tx = 1, ty =1) STATE

$C[1][1]$ | row =1, col =1

$9 \times 0 + 10 \times 3 = 30$

CValue = 8 + 30 = **38**



!

Each thread adds to its running Cvalue.
Notice the **same shared memory tile**
is used by all 4 threads, that's the reuse.

Lets focus on multiple threads within a block — Computation
block = 0: thread = (0,0), thread = (1,0), thread = (0,1), thread = (1,1)

A (4x6)

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)

1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6

Load into shared memory

5	6	5	0
11	12	0	5

Iteration = 2 (Final tile)

C (4x4)

(1,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Thread (tx = 0, ty =0) STATE

C[0][0] | row =0, col =0

5x5 + 6x0 = 25

CValue = 10 + 25 = 35

Thread (tx = 1, ty =0) STATE

C[0][1] | row =0, col =1

5x0 + 6x5 = 30

CValue = 14 +30= 44

Thread (tx = 0, ty =1) STATE

C[1][0] | row =1, col =0

11x5 + 12x0 = 55

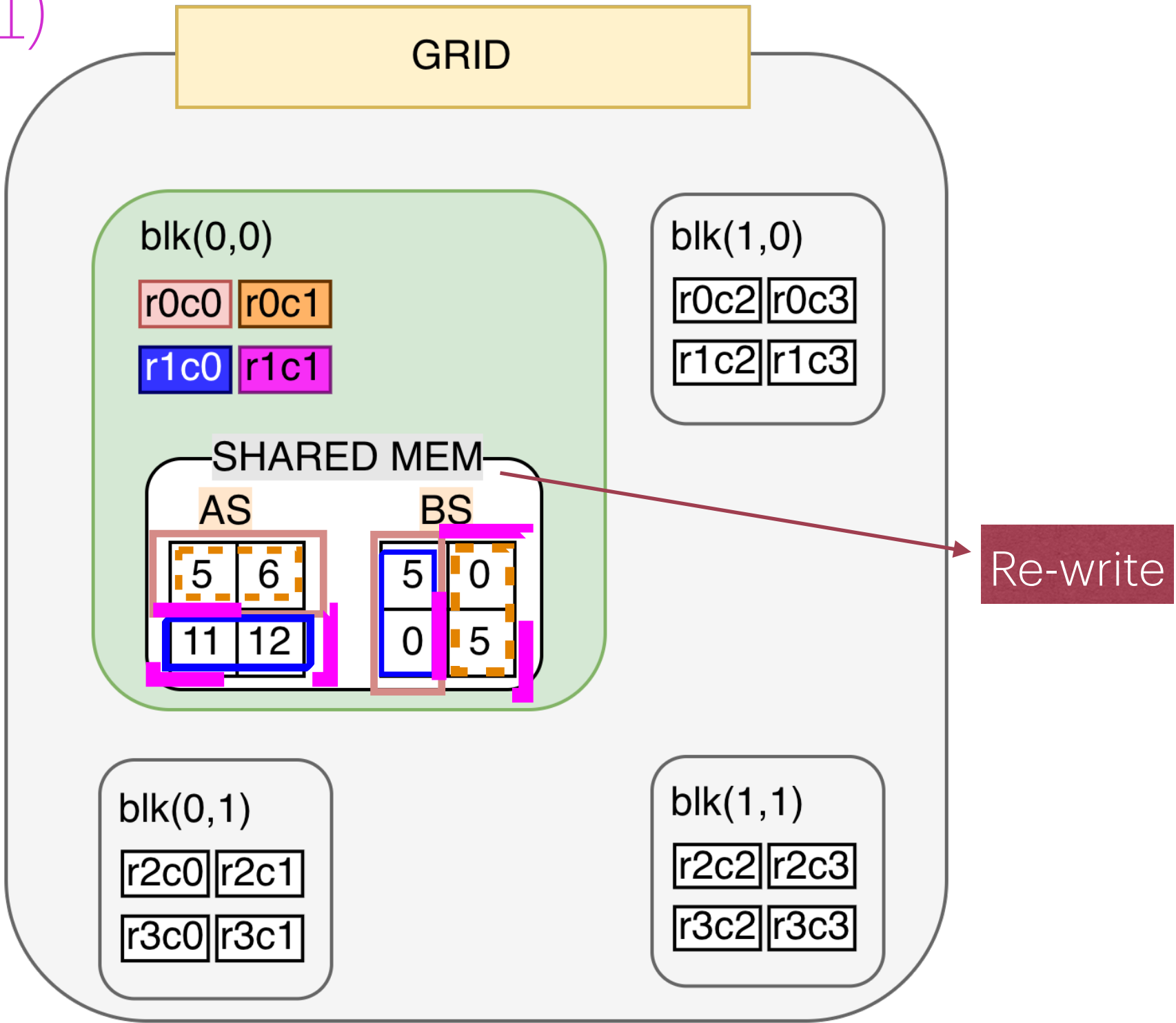
CValue = 34 + 55 = 89

Thread (tx = 1, ty =1) STATE

C[1][1] | row =1, col =1

11x0 +12x5 = 60

CValue = 38 + 60 = 98



! Last partial sum added. Each thread now holds the complete dot product for its cell.

Lets focus on multiple threads within a block — Write the result
block = 0: thread = (0,0), thread = (1,0), thread = (0,1), thread = (1,1)

A (4x6)					
1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24

B (6x4)			
1	0	2	0
0	1	0	2
3	0	4	0
0	3	0	4
5	0	6	0
0	5	0	6

!

All threads write at the same time.
No coordination needed, each thread owns a unique cell.
C[0][0]=35, C[0][1]=44, C[1][0]=89, C[1][1]=98

C (4x4)			
35	44	(0,2)	(0,3)
89	98	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

!

All 4 BLOCKS — RUNNING IN PARALLEL
All 4 blocks launch simultaneously the moment the kernel starts. Each block independently loads its own tiles from A and B into its own shared memory, runs the same 3-iteration outer loop, and accumulates C values. No block knows or cares what the others are doing.